

# AI StudyBuddy (LearnMate)

## Enterprise Technical Architecture & Comprehensive Implementation Guide

---

A Deep-Dive Architectural Blueprint, Data Modeling Specification, API Reference,  
and Source Implementation Guide for an AI-Powered Learning Backend.

**System Version:** 2.5.0-RELEASE

**Core Stack:** Node.js, Express.js, MongoDB Atlas, Mongoose ODM

**AI Engine:** Google Gemini 2.5 Flash Generative AI

**Security:** JWT Auth & Role-Based Access Control (RBAC)

**Date:** September 2026

# Table of Contents & Executive Overview

## Document Structure & Chapter Index

- 1. Chapter 1: Executive Overview & Problem Context ..... Page 3
- 2. Chapter 2: Scenario-Based Case Study (Rahul's Academic Persona) ..... Page 4
- 3. Chapter 3: Software & Hardware Infrastructure Requirements ..... Page 5
- 4. Chapter 4: Environment & Operational Configuration ..... Page 6
- 5. Chapter 5: Architectural Foundations & Component Layering ..... Page 7
- 6. Chapter 6: Model-View-Controller (MVC) Structural Analysis ..... Page 8
- 7. Chapter 7: Entity-Relationship (ER) Architecture & Relational Model ..... Page 9
- 8. Chapter 8: Data Models — User Schema & Security Hooks ..... Page 10
- 9. Chapter 9: Data Models — Material Schema & Embedded AI Store ..... Page 11
- 10. Chapter 10: Security Middleware & Token Verification Engine ..... Page 12
- 11. Chapter 11: File Parsing & Storage Pipeline (Multer Integration) ..... Page 13
- 12. Chapter 12: Google Gemini Generative AI Service Layer ..... Page 14
- 13. Chapter 13: Application Gateway & Controller Implementations ..... Page 15
- 14. Chapter 14: RESTful API Route Interface & Complete Endpoint Catalog ..... Page 16
- 15. Chapter 15: Detailed API Specifications — Authentication & Management ..... Page 17
- 16. Chapter 16: Detailed API Specifications — AI Content Generation ..... Page 18
- 17. Chapter 17: User Flow & Application Lifecycle Engineering ..... Page 19
- 18. Chapter 18: System Deployment, Testing Protocols & Resource Index ..... Page 20

# 1. Executive Overview & Platform Vision

In contemporary higher education, students are flooded with an overwhelming volume of instructional materials distributed across heterogeneous channels, including PDF textbook chapters, digital slides, raw lecture transcripts, and markdown study notes. The primary operational bottleneck in learning is no longer access to information, but rather the intensive cognitive effort required to process, summarize, index, and continuously evaluate comprehension of these materials.

**AI StudyBuddy** (internally designated as *LearnMate*) is a state-of-the-art backend ecosystem built to eliminate academic administration overhead through automated, AI-driven content transformation. By utilizing modern web application frameworks and Google's flagship Gemini Generative AI technology, AI StudyBuddy delivers automated educational workflows directly to client interfaces.

## Core Platform Pillars

- **Automated Synthesis:** High-density long-form document extraction and contextual summarization using natural language processing.
- **Active Recall Generation:** Automatic generation of study flashcards containing question-and-answer pairs extracted from source material.
- **Continuous Knowledge Assessment:** Intelligent dynamic quiz generation with multiple-choice options, distractors, and verifiable target answers.
- **Customized Temporal Planning:** Generation of study schedules conditioned on target examination dates and available student hours.
- **Enterprise-Grade Security:** Cryptographically signed JWT session control paired with bcrypt key derivation and Role-Based Access Control (RBAC).

By separating authentication, business domain logic, database operations, and external AI integrations into modular micro-architectural layers, the backend guarantees high throughput, low latency response handling, and fault tolerance across diverse academic workloads.

## 2. Scenario-Based Case Study: Rahul's Workflow

To validate the functional requirements and architectural objectives of the platform, the system was modeled against a high-stress student environment. This section details the operational scenario of a target user persona.

### User Profile: Rahul (2nd Year Computer Science Student)

**Academic Setting:** Rahul is currently balancing 6 core engineering subjects, active internship interviews, and practical laboratory assignments. His study materials consist of hundreds of pages of unformatted PDF notes, instructor slide decks, and code repositories.

### 2.1 The Friction Points & Operational Challenges

- **Information Overload & Cognitive Fatigue:** Reading 80+ pages of dense theoretical text prior to late-night revision sessions leads to poor memory retention and severe burnout.
- **Inefficient Manual Summarization:** Spending 3 to 4 hours manually creating revision bullet points reduces time available for active problem solving.
- **Lack of Self-Assessment Tools:** Preparing mock examination questions manually is time-consuming and often introduces bias toward topics already mastered.
- **Fragmented Tooling:** Switching between note-taking apps, flashcard tools, digital calendars, and file drives degrades focus and productivity.

### 2.2 The AI StudyBuddy Solution Path

With AI StudyBuddy, Rahul uploads raw lecture text or file documents into a unified backend interface. Within seconds, the backend processes the input through Google Gemini AI to return:

1. A concise key-concept summary for immediate pre-exam reviewing.
2. A structured set of active-recall flashcards.
3. A dynamic self-assessment quiz to evaluate knowledge gaps.
4. An actionable 7-day revision timeline fitted to his daily free hours.

## 3. Software & Hardware Infrastructure Requirements

Deploying and running the AI StudyBuddy backend requires a well-defined runtime environment to handle concurrent REST API requests, heavy asynchronous I/O operations, and database query executions efficiently.

### 3.1 Software Technology Stack

| Component             | Technology Selected      | Version / Specification | Role in System Architecture                                             |
|-----------------------|--------------------------|-------------------------|-------------------------------------------------------------------------|
| Runtime Environment   | Node.js                  | v16.0.0 or higher       | Executes non-blocking, event-driven asynchronous server logic.          |
| Package Manager       | npm                      | v8.0.0 or higher        | Manages software dependencies and executable build scripts.             |
| Web Framework         | Express.js               | v4.18.2                 | Handles HTTP routing, middleware pipelines, and API responses.          |
| Database Engine       | MongoDB                  | v6.0+ / Atlas Cloud     | Document NoSQL store holding collections for users and study materials. |
| Object Modeling (ODM) | Mongoose                 | v8.0.0+                 | Enforces strict data schemas, validation rules, and queries.            |
| AI Gateway Engine     | Google Gemini AI API     | Gemini 2.5 Flash        | Processes natural language text for AI content generation.              |
| Testing Harness       | Postman / Thunder Client | Latest Release          | Executes API endpoint validation and token assertion checks.            |

### 3.2 Hardware Specifications

#### Recommended Server Resources

- **Processor (CPU):** Multi-core x86\_64 or ARM processor (Intel Core i5 8th Gen+, AMD Ryzen 5, or AWS Graviton instances).
- **System Memory (RAM):** Minimum 8 GB required; 16 GB recommended for running concurrent MongoDB daemons, Node execution threads, and file upload parsing buffers.
- **Storage Workspace:** Minimum 1 GB available disk space for application files, Node modules, temporary upload buffers, and server logging files.

# 4. Environment & Operational Configuration

The AI StudyBuddy application uses environment variables to isolate sensitive credentials, secret keys, and database connections from source code. This approach ensures security across development, staging, and production environments.

## 4.1 Environment Variable Schema (.env)

A `.env` file must be created at the root directory of the project prior to server initialization. The table below defines every required environment key.

| Variable Key Name | Type / Format          | Example Value             | Functional Requirement                                                |
|-------------------|------------------------|---------------------------|-----------------------------------------------------------------------|
| PORT              | Numeric Port           | 5000                      | Specifies the network port on which the Express server listens.       |
| MONGO_URI         | Connection String      | mongodb+srv://...         | Authenticated connection URI for the target MongoDB database.         |
| JWT_SECRET        | String (Cryptographic) | super_secret_key_99       | Used for signing generic JSON Web Tokens across legacy handlers.      |
| JWT_ACCESS_SECRET | String (Cryptographic) | access_token_secret_#2026 | Used by <code>auth.js</code> middleware to verify user access tokens. |
| GEMINI_API_KEY    | Alphanumeric Key       | AIzaSyD-x9812A...         | Google Cloud API Key with access to Google Gemini models.             |
| CLIENT_URL        | URL String             | http://localhost:3000     | Configures CORS policies to allow requests from the React frontend.   |

## 4.2 Root Project Directory Setup

```
AI-STUDYBUDDY/
├── .env
├── .gitignore
├── index.js
├── package.json
├── package-lock.json
├── README.md
└── src/
    ├── controllers/
    │   ├── adminController.js
    │   ├── authController.js
    │   └── materialController.js
    ├── middleware/
    │   ├── auth.js
    │   └── upload.js
    ├── models/
    │   ├── Material.js
    │   └── User.js
    ├── routes/
    │   ├── admin.js
    │   ├── auth.js
    │   └── materials.js
    └── utils/
        ├── db.js
        ├── gemini.js
        └── tokens.js
```

## 5. Architectural Foundations & Component Layering

The AI StudyBuddy system is built on a modular, multi-layered architecture designed to support high throughput, loose coupling, and maintainability.

### 5.1 Architectural Flow Diagram Representation

#### System Request Lifecycle & Layer Interactions

- 1. Client Tier (React Frontend):** Sends HTTP REST requests containing JSON payloads or multi-part file uploads, accompanied by JWT Authorization Bearer headers.
- 2. Gateway Layer (Express Server):** Handles CORS policies, parses JSON payloads, and routes requests to appropriate endpoints.
- 3. Security Tier (Middleware Engine):** Intercepts protected requests, verifies JWT access tokens, attaches user identities to request objects, and enforces RBAC restrictions.
- 4. Business Logic Tier (Controllers):** Processes user inputs, executes document parsing, invokes database queries, and manages interactions with external services.
- 5. Integration Tier (Google Gemini Service):** Formulates contextual prompts, transmits API calls to Gemini 2.5 Flash, and parses structured AI responses.
- 6. Data Persistence Tier (MongoDB Atlas):** Stores user accounts, uploaded materials, and AI-generated assets as structured documents via Mongoose ODM.

### 5.2 Architectural Principles

- **Single Responsibility Principle:** Each controller, utility module, and middleware component handles a single specific task within the application lifecycle.
- **Asynchronous Non-Blocking Execution:** Database reads, external API calls, and file system operations use `async/await` patterns to maximize event loop performance.
- **Stateless Authentication:** User session state is encoded inside cryptographically signed JWT tokens, eliminating the need for server-side session stores.

## 6. Model-View-Controller (MVC) Structural Analysis

The AI StudyBuddy backend follows the Model-View-Controller (MVC) architectural pattern to enforce a clean separation of concerns between data storage, processing logic, and response formatting.

### 6.1 Model Layer (Data Management)

The **Model Layer** is built using Mongoose schemas on top of MongoDB collections. It manages data structure definitions, field validation rules, default values, schema hooks, and database operations. By encapsulating database interactions within model files, business controllers can manipulate data objects without writing raw database queries.

### 6.2 Controller Layer (Business Domain Logic)

The **Controller Layer** contains the central business logic of the application. Controllers receive request objects from express routes, validate incoming input data, coordinate interactions between models and external services (like Google Gemini AI), handle runtime exceptions, and format appropriate HTTP responses.

### 6.3 View Layer (API REST Response Representation)

In a headless RESTful architecture, the traditional HTML View layer is replaced by a structured JSON API response layer. The Express application formats responses as standard JSON payloads containing status codes, status messages, data models, and error descriptions for consumption by client interfaces.

| MVC Layer  | System Component Directory            | Primary Responsibilities                                                                     |
|------------|---------------------------------------|----------------------------------------------------------------------------------------------|
| Model      | src/models/*.js                       | Defines database schemas, relationships, constraints, and hook functions.                    |
| Controller | src/controllers/*.js                  | Executes business logic, input checks, external API integrations, and responses.             |
| View (API) | src/routes/*.js + JSON payload output | Defines public API contracts and serializes database objects into structured JSON responses. |

## 7. Entity-Relationship (ER) Architecture

Although MongoDB is a document-oriented NoSQL database, AI StudyBuddy maintains clear logical relationships between its core entities to ensure data integrity and efficient query performance.

### 7.1 Entity Specifications

- **User Entity:** Stores authentication credentials, profile data, created dates, and authorization roles (Student vs. Admin).
- **Material Entity:** Acts as the parent container for a user's uploaded study content. Holds extracted text, file metadata, and references to generated AI assets.
- **Summary Sub-document/Entity:** Plaintext or Markdown summary text generated by Gemini AI, linked to a target Material entity.
- **Flashcard Sub-document Entity Array:** Array of question-and-answer objects associated with a specific study material.
- **Quiz Sub-document Entity Array:** Array of multiple-choice question objects containing distractor choices and correct answer keys.
- **Study Plan Sub-document/Entity:** Personalized study schedule generated from user goals, available study hours, and target timelines.

### 7.2 Cardinality & Relationship Mapping

#### Logical Relationship Mapping

- **User to Study Material (One-to-Many):** A single User can upload multiple study materials. Each material document stores its parent user's ObjectId in its `user` reference field.
- **Study Material to Summary (One-to-One):** Each study material contains one generated summary block.
- **Study Material to Flashcards (One-to-Many):** A single study material document can contain an array of multiple flashcard objects.
- **Study Material to Quiz Questions (One-to-Many):** A study material document contains an array of multiple-choice quiz items.
- **User to AI Resources (One-to-Many):** All generated learning assets are owned by the User who uploaded the source material, ensuring data privacy and access control.

## 8. Data Models — User Schema & Security Hooks

---

The `User` schema manages user credentials, authorization roles, and security hooks. Passwords are automatically hashed prior to database persistence using `bcryptjs`.

## 8.1 Source Code: src/models/User.js

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

/**
 * User Schema definition for AI StudyBuddy
 * Tracks identity, credentials, role-based security, and timestamp metadata.
 */
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, "Name field is mandatory"],
      trim: true,
    },
    email: {
      type: String,
      required: [true, "Email field is mandatory"],
      unique: true,
      lowercase: true,
      trim: true,
    },
    password: {
      type: String,
      required: [true, "Password field is mandatory"],
    },
    role: {
      type: String,
      enum: ["student", "admin"],
      default: "student",
    },
  },
  {
    timestamps: true,
  }
);

// Pre-save middleware hook for password encryption
userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) {
    return next();
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Instance method to compare input password with stored hash
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model("User", userSchema);
```

## 9. Data Models — Material Schema & Embedded AI Store

---

The `Material` schema serves as the primary data document for user learning materials. It stores uploaded content along with embedded schemas for summaries, flashcards, quizzes, and study plans.

## 9.1 Source Code: src/models/Material.js

```
const mongoose = require("mongoose");

/**
 * Material Schema definition for AI StudyBuddy
 * Stores uploaded content alongside AI-generated learning resources.
 */
const materialSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    title: {
      type: String,
      required: [true, "Material title is required"],
      trim: true,
    },
    content: {
      type: String,
      required: [true, "Material text content is required"],
    },
    filename: {
      type: String,
      default: null,
    },
    summary: {
      type: String,
      default: null,
    },
    flashcards: [
      {
        question: { type: String, required: true },
        answer: { type: String, required: true },
      },
    ],
    quiz: [
      {
        question: { type: String, required: true },
        options: [{ type: String, required: true }],
        answer: { type: String, required: true },
      },
    ],
    studyPlan: {
      type: String,
      default: null,
    },
  },
  {
    timestamps: true,
  }
);

module.exports = mongoose.model("Material", materialSchema);
```

## 10. Security Middleware & Token Verification

API routes are protected using JSON Web Token (JWT) verification middleware. The middleware validates token signatures, checks expiration times, and enforces Role-Based Access Control (RBAC).

### 10.1 Source Code: src/middleware/auth.js

```
const jwt = require("jsonwebtoken");

/**
 * Authentication Middleware: Verifies JWT token in Bearer header.
 */
const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Not authenticated, token missing" });
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_ACCESS_SECRET || process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ message: "Invalid or expired authorization token" });
  }
};

/**
 * Authorization Middleware: Restricts route access to Admin users.
 */
const adminOnly = (req, res, next) => {
  if (!req.user || req.user.role !== "admin") {
    return res.status(403).json({ message: "Access forbidden: Administrative permissions required" });
  }
  next();
};

module.exports = { protect, adminOnly };
```

# 11. File Parsing & Storage Pipeline

The upload middleware uses `Multer` to handle file uploads securely. It validates file extensions, enforces size limits, and stores files to disk before text extraction.

## 11.1 Source Code: `src/middleware/upload.js`

```
const multer = require("multer");
const path = require("path");

// Storage configuration for Multer disk engine
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, "uploads/");
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + "-" + Math.round(Math.random() * 1e9);
    cb(null, uniqueSuffix + path.extname(file.originalname));
  },
});

// File filter to restrict allowed document types
const fileFilter = (req, file, cb) => {
  const allowedExtensions = [".txt", ".md", ".pdf"];
  const ext = path.extname(file.originalname).toLowerCase();

  if (allowedExtensions.includes(ext)) {
    cb(null, true);
  } else {
    cb(new Error("File format unsupported. Only .txt, .md, and .pdf files are allowed."), false);
  }
};

// Multer upload middleware instance (5 MB file size limit)
const upload = multer({
  storage: storage,
  fileFilter: fileFilter,
  limits: { fileSize: 5 * 1024 * 1024 },
});

module.exports = upload;
```

## 12. Google Gemini Generative AI Service Layer

The AI integration utility connects the application to Google's `gemini-2.5-flash` model using the official `@google/generative-ai` SDK, enabling natural language generation for study resources.

### 12.1 Source Code: `src/utils/gemini.js`

```
const { GoogleGenerativeAI } = require("@google/generative-ai");

// Initialize Google Generative AI SDK with API key from environment
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
const model = genAI.getGenerativeModel({ model: "gemini-2.5-flash" });

/**
 * Transmits natural language prompts to Google Gemini API
 * @param {string} prompt - Formatted text prompt for AI processing
 * @returns {Promise} Generative text response from model
 */
const askGemini = async (prompt) => {
  try {
    const result = await model.generateContent(prompt);
    const response = await result.response;
    return response.text();
  } catch (error) {
    console.error("Gemini AI API Error:", error.message);
    throw new Error("Failed to process content with AI service provider.");
  }
};

module.exports = { askGemini };
```

## 13. Application Gateway & Server Entry Point

The `index.js` file initializes the Express application, configures middleware, establishes the MongoDB database connection, registers API routes, and starts the server.

### 13.1 Source Code: `index.js`

```
require("dotenv").config();
require("express-async-errors");

const express = require("express");
const cors = require("cors");
const connectDB = require("../src/utils/db");

const app = express();

// Global Express Middleware
app.use(express.json());
app.use(cors({ origin: process.env.CLIENT_URL || "http://localhost:3000" }));

// Primary API Route Declarations
app.use("/api/auth", require("../src/routes/auth"));
app.use("/api/materials", require("../src/routes/materials"));
app.use("/api/admin", require("../src/routes/admin"));

// Health Check Endpoint
app.get("/", (req, res) => {
  res.json({ status: "Online", message: "AI StudyBuddy REST API Service is Running" });
});

// Global Centralized Error Handling Middleware
app.use((err, req, res, next) => {
  console.error("Global Error Handler Catch:", err.message);
  const status = err.status || 500;
  res.status(status).json({
    error: true,
    message: err.message || "An unexpected internal server error occurred",
  });
});

const PORT = process.env.PORT || 5000;

// Initialize Database Connection and Start Server
connectDB().then(() => {
  app.listen(PORT, () => {
    console.log(`AI StudyBuddy Server operational on port ${PORT}`);
  });
});
```

# 14. RESTful API Route Interface Catalog

The system exposes structured REST API endpoints for user authentication, material management, AI resource generation, and administrative system monitoring.

| HTTP Method | API Route Endpoint            | Authentication / Role  | Functional Overview                                                            |
|-------------|-------------------------------|------------------------|--------------------------------------------------------------------------------|
| POST        | /api/auth/register            | Public                 | Registers a new user account with hashed password and returns access tokens.   |
| POST        | /api/auth/login               | Public                 | Authenticates user credentials and generates JWT access/refresh tokens.        |
| POST        | /api/materials/upload         | Protected (User)       | Parses uploaded file or text input and saves raw study material to MongoDB.    |
| GET         | /api/materials                | Protected (User)       | Retrieves a list of all study materials owned by the authenticated user.       |
| POST        | /api/materials/:id/summarize  | Protected (User)       | Calls Gemini AI to generate and persist a summary for the specified material.  |
| POST        | /api/materials/:id/flashcards | Protected (User)       | Generates active-recall flashcard pairs from material content using Gemini AI. |
| POST        | /api/materials/:id/quiz       | Protected (User)       | Generates a multiple-choice quiz with correct answer keys using Gemini AI.     |
| POST        | /api/materials/:id/study-plan | Protected (User)       | Generates a personalized daily revision schedule based on time preferences.    |
| GET         | /api/admin/users              | Protected (Admin Only) | Returns a list of registered users for administrative monitoring.              |

## 15. Detailed API Specifications — Authentication & Management

### 15.1 User Registration Endpoint

**Endpoint:** POST /api/auth/register | **Access:** Public

**Request Body (JSON):**

```
{
  "name": "Rahul Kumar",
  "email": "rahul@example.com",
  "password": "SecurePassword123",
  "role": "student"
}
```

**Success Response (201 Created):**

```
{
  "message": "Registered successfully",
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "643f6cd2173d7853fc8b3842",
    "name": "Rahul Kumar",
    "role": "student"
  }
}
```

### 15.2 Material Upload Endpoint

**Endpoint:** POST /api/materials/upload | **Access:** Protected (Bearer Token Required)

**Request Body (Multipart Form-Data or JSON):**

```
{
  "title": "Node.js Architecture Overview",
  "content": "Node.js is an open-source, cross-platform JavaScript runtime environment..."
}
```

**Success Response (201 Created):**

```
{
  "message": "Material uploaded successfully",
  "material": {
    "_id": "6a3f738ab4b7ea076b12d3ab",
    "user": "643f6cd2173d7853fc8b3842",
    "title": "Node.js Architecture Overview",
    "createdAt": "2026-09-25T10:15:30.000Z"
  }
}
```

## 16. Detailed API Specifications — AI Content Generation

### 16.1 Flashcard Generation Endpoint

**Endpoint:** POST /api/materials/:id/flashcards | **Access:** Protected

**Success Response (200 OK):**

```
{
  "flashcards": [
    {
      "question": "What is Node.js?",
      "answer": "Node.js is an open-source, cross-platform JavaScript runtime built on Google V8 engine"
    },
    {
      "question": "How does Node.js handle concurrent connections?",
      "answer": "Node.js uses an event-driven, non-blocking I/O model running on a single-threaded Event Loop"
    }
  ]
}
```

### 16.2 Quiz Generation Endpoint

**Endpoint:** POST /api/materials/:id/quiz | **Access:** Protected

**Success Response (200 OK):**

```
{
  "quiz": [
    {
      "question": "Which model best describes Node.js's I/O processing?",
      "options": [
        "A) Multi-threaded blocking I/O",
        "B) Single-threaded blocking I/O",
        "C) Multi-threaded non-blocking I/O",
        "D) Single-threaded non-blocking I/O"
      ],
      "answer": "D"
    }
  ]
}
```

# 17. User Flow & Application Lifecycle Engineering

This chapter outlines the end-to-end user workflow within the AI StudyBuddy platform, tracing user interactions from registration to resource generation and management.

## 17.1 User Flow Lifecycle Sequence

### Step-by-Step User Journey

#### 1. Authentication Phase:

- The user submits credentials via the registration or login screen.
- The backend authenticates credentials and returns cryptographically signed JWT tokens.
- The client stores tokens securely for authorizing subsequent REST API calls.

#### 2. Content Ingestion Phase:

- The user uploads study documents or inputs raw notes into the dashboard.
- Multer processes file uploads, validates document types, and extracts text content.
- The extracted content is stored in the MongoDB database linked to the user's ID.

#### 3. AI Processing Phase:

- The user selects an AI feature: Summary, Flashcards, Quiz, or Study Plan.
- The controller constructs a contextual prompt and invokes the Google Gemini API.
- Gemini AI generates the structured response, which is returned to the client and saved in MongoDB.

#### 4. Revision & Dashboard Management Phase:

- The user views generated learning assets directly in the client application.
- All saved materials and generated resources can be accessed later from the dashboard history.

## 18. System Deployment, Testing & Resource Index

This final chapter covers deployment procedures, testing commands, and project links for reference.

### 18.1 Local Execution & Server Deployment

#### Command Execution Sequence

1. Install all system dependencies:

```
npm install
```

2. Launch the development server with hot-reloading:

```
npm run dev
```

3. Start the production server:

```
npm start
```

### 18.2 API Testing Protocol (Postman Verification)

- **Auth Headers:** Set header `Authorization: Bearer <YOUR_ACCESS_TOKEN>` on protected routes.
- **Form Uploads:** Send multi-part form-data with the `file` key for document uploads.
- **Assertions:** Verify `200 OK` or `201 Created` HTTP response codes and JSON schema structure.

### 18.3 Project Media & Asset Index

#### Media & External Artifact Links

Demo Video: [Open Google Drive Demo Video](#)

Execution Screen Recording: [Open Google Drive Screen Recording](#)

---

— END OF TECHNICAL SPECIFICATION & IMPLEMENTATION MANUAL —